

іТМО

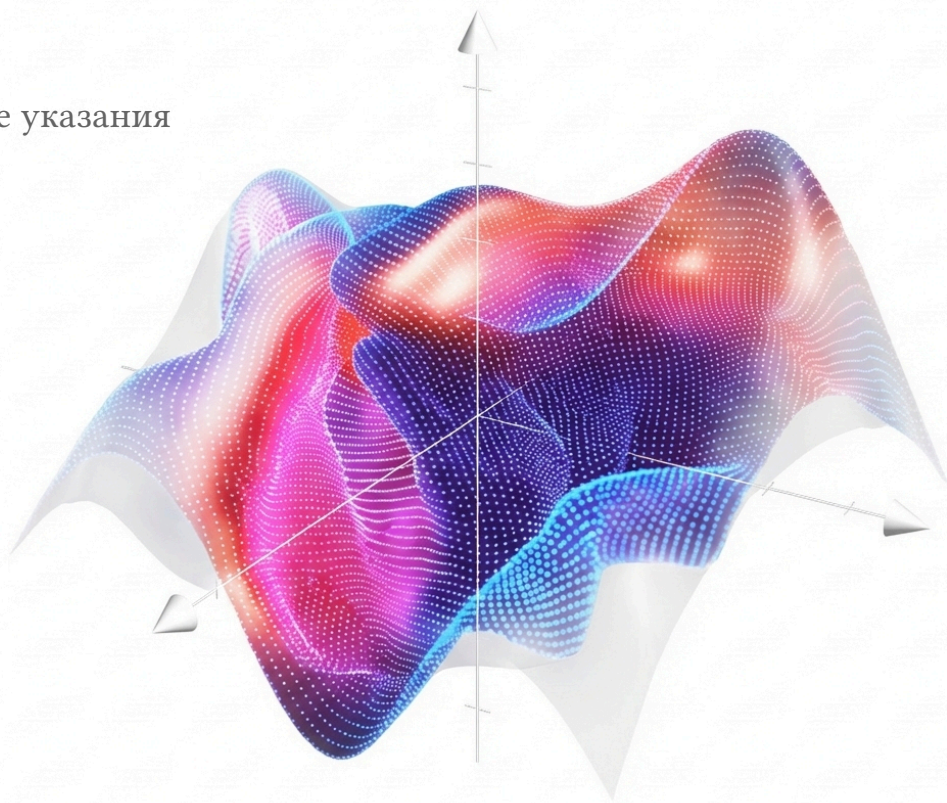
И

Лабораторные работы

Продвинутые методы оптимизации

Методические указания

2026



1 Общие положения и структура репозитория

Целью данных указаний является **стандартизация процесса сдачи лабораторных работ** для обеспечения воспроизводимости исследований и удобства код-ревью.

1.1 Работа с репозиторием

Процесс подготовки

- **Fork**: Вы выполняете fork основного репозитория курса (ориджин) в свой личный аккаунт на Git-хостинге (GitHub/GitLab).
- **Клонирование**: Клонируете свой форк на локальную машину.
- **Ветвление**: Все изменения ведутся в отдельной ветке. Название ветки должно соответствовать шаблону: `lab<номер>`.

1.2 Структура папок

Ниже приведена обязательная иерархия файлов в вашем репозитории:

Структура репозитория курса

```
репозиторий-курса
├── lab_01                                # Папка для первой лабораторной работы
│   ├── fish.csv                         # Датасет
│   ├── report.md                       # Отчет по работе
│   ├── research.ipynb                  # Jupyter/Colab ноутбук
│   ├── src/                            # Папка с исходным кодом
│   │   ├── model.py                    # Реализация методов
│   │   └── utils.py                    # Вспомогательные функции
│   └── requirements.txt                 # Зафиксированные зависимости
├── lab_02/
├── README.md                           # Основное описание репозитория
└── .gitignore
```

Убедитесь, что все зависимости зафиксированы в `requirements.txt`, чтобы проверяющий мог развернуть вашу среду без ошибок.

2 Требования к оформлению кода и исследовани

2.1 Разделение логики и экспериментов

Код должен быть модульным, чтобы его можно было переиспользовать и тестировать.

src/ (Реализация)

Содержит чистый исходный код. **Не используйте ноутбуки для хранения бизнес-логики или реализации алгоритмов.**

Пример реализации (src/model.py)

```
import numpy as np

class LinearRegression:
    def __init__(self, lr: float = 0.01):
        self.lr = lr
        self.weights = None

    def fit(self, X: np.ndarray, y: np.ndarray) -> None:
        # Здесь код обучения
        pass

    def predict(self, X: np.ndarray) -> np.ndarray:
        # Здесь код предсказания
        pass
```

research.ipynb (Исследование)

Содержит визуализации, графики, выводы и демонстрацию работы функций из папки `src/`.
Импортируйте код как пакет.

Пример ячейки в ноутбуке

```
# Импорт собственного модуля
import sys
sys.path.append('src') # или установка пакета в режиме разработки
from model import LinearRegression

# Далее следует код эксперимента с графиками
```

Key Insight

Разделение на `src/` и `notebooks` позволяет легко версионировать код и избегать огромных диффов в Git при изменении только визуализаций.

2.2 Отчет по работе

Отчет **не должен содержать тонну кода**. Это документ для преподавателя, описывающий концептуальную часть: что вы сделали и почему.

Формат: `README.md` в папке работы (или прямая ссылка на `report.pdf` в той же папке).

Обязательное содержание отчета

4. **Результаты экспериментов:**

- Метрики (MSE, MAE и т.д.).
- Графики

5. **Выводы:** Интерпретация полученных результатов.

Совет по оформлению

Используйте относительные ссылки в Markdown (например, `[Код реализации](./src/model.py)`), чтобы из вашего отчета можно было в один клик перейти к нужному файлу прямо в интерфейсе GitHub/GitLab.

3 Воспроизводимость результатов

Определение:

Воспроизводимость — это способность другого разработчика (или проверяющего) запустить ваш код и получить те же результаты, что указаны в отчете.

3.1 Фиксация случайных seed'ов

В начале каждого ноутбука и в корне скриптов обучения необходимо фиксировать зерно генератора случайных чисел (Random Seed).

Пример фиксации для Python/NumPy

```
import random
import numpy as np

def seed_everything(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    # Если используется PyTorch:
    # torch.manual_seed(seed)

seed_everything()
```

3.2 Управление зависимостями

Недостаточно просто сказать «установите numpy». Нужна жесткая фиксация версий. Выберите один из вариантов:

Вариант А — Poetry (Рекомендуется)

- **Инициализация:** `poetry init` в корне.
- **Добавление пакетов:** `poetry add numpy pandas scikit-learn`.
- **Что коммитить:** `pyproject.toml` и `poetry.lock`.
- **Инструкция для проверки:** `poetry install`.

Вариант Б — Pip + VirtualEnv

- **Активация:** создание и активация `venv`.
- **Фиксация:** `pip freeze > requirements.txt`.
- **Что коммитить:** `requirements.txt`.
- **Инструкция для проверки:** `pip install -r requirements.txt`.

Вариант В — Docker

Самый надежный способ. Напишите `Dockerfile`, который изолирует зависимости и версию ОС. Это гарантирует 100% воспроизводимость.

Вариант Г — Conda/Anaconda

- **Создание окружения:** `conda create --name lab_env python=3.9`.
- **Экспорт:** `conda env export > environment.yml`.
- **Что коммитить:** `environment.yml`.
- **Инструкция для проверки:** `conda env create -f environment.yml`.

Отсутствие файла с зависимостями — автоматический минус к оценке за лабораторную работу.

4 Процесс сдачи работы (CI/CD и ревью)

Процесс сдачи автоматизирован для обеспечения максимальной прозрачности и объективности оценки.

4.1 Локальное тестирование (перед коммитом)

Перед отправкой изменений (push) в удаленный репозиторий обязательно убедитесь, что:

- **Чистое выполнение:** Ноутбук выполняется от начала до конца без ошибок (`Kernel` -> `Restart & Run All`).

- **Стиль кода:** Код соответствует стандартам оформления (используйте линтеры, например, `black` или `flake8`).

4.2 CI/CD (Непрерывная интеграция)

При создании Pull Request (PR) в основной репозиторий автоматически запускается пайплайн проверки.

Цель CI

Проверить, что ваш код работает в «чистом» окружении, а не только на вашей локальной машине. Это гарантирует корректность зависимостей и отсутствие ошибок при выполнении ноутбука.

Типовые шаги CI (.github/workflows/ci.yml)

1. **Checkout:** Клонирование вашего репозитория на агент сборки.
2. **Setup:** Установка Python и зависимостей из `poetry.lock` или `requirements.txt`.
3. **Lint:** Автоматическая проверка стиля кода линтером.
4. **Headless Run:** Запуск ноутбука в фоновом режиме для проверки работоспособности.
5. **Test:** Прогон юнит-тестов (при их наличии).

IMPORTANT: Статус инфраструктуры

Для работы автоматизации необходима предварительная настройка серверов (воркеров). На текущий момент этап поиска и отладки воркеров находится в статусе **WIP (Work In Progress)**.

Студентам не нужно об этом беспокоиться, но важно понимать: автоматическая проверка станет доступна только после завершения этой настройки.

До момента запуска CI/CD проверка будет осуществляться преподавателем вручную в процессе код-ревью.

4.3 Pull Request и ревью

После успешного локального тестирования и пуша в свою ветку, переходите к созданию запроса на слияние (Pull Request).

Этапы Pull Request

1. **Создание PR:** Из вашей ветки (`lab01_ivanov`) в ветку `main` основного репозитория.
2. **Описание:** В описании PR кратко укажите, что именно было реализовано.
3. **Проверка CI:** Статус CI должен гореть **зеленым**. Если пайплайн упал, исправьте замечания до запроса ревью.

Очное ревью: После того как код принят автоматикой и проверен асинхронно, назначается очная встреча. На ней вы объясняете ключевые моменты реализации и отвечаете на вопросы преподавателя.

5 Система оценивания и баллы

За выполнение всех лабораторных работ в рамках курса можно получить от **50 до 80 баллов**.

Критерии оценки

Итоговый балл за работу формируется на основе следующих факторов:

- **Качество отчета:** полнота описания, наличие графиков и глубина выводов.
- **Защита:** понимание сути реализованных методов и ответов на вопросы.
- **Качество кода:** отсутствие «спагетти-кода» (небрежный код снижает балл).
- **Своевременность:** срок сдачи — **2 недели** с момента получения задания.

5.1 Сроки и штрафы

Срок сдачи фиксируется по времени отправки работы в специальный ящик. При нарушении дедлайна начисляется штраф: **-25% от баллов за каждую неделю просрочки**.

5.2 Командная работа и репутация

Работа в группах

Лабораторные работы допускается выполнять командой из **2 (иногда 3) человек**.

- При защите основной работы баллы начисляются всем участникам в полном объеме (без деления).
- В случае выполнения дополнительных заданий (тендеров) баллы делятся между участниками.

Система репутации

Для разрешения спорных ситуаций (например, в тендерах) используется показатель репутации. Её можно заработать за:

- Активное участие в воркшопах.
- Своевременное выполнение и успешную защиту лабораторных работ.

Удачи в выполнении!

Соблюдение этих простых правил значительно ускорит процесс проверки и поможет вам выработать навыки профессиональной разработки.